INTERFACEGUARD

Object-Oriented Interface Erosion Audit

Repository: **order-service** • URL: sample-repositories/order-service • Generated: 2026-10-26 06:58 UTC

1. Executive Summary & Project Erosion Index

Project Erosion Index	**81.5 / 100**	**Status:**	**Healthy**
Analyzed Commits:	6	**Tracked Interfaces:**	2
ISP Violations:	6	**Dependency Cycles:**	2

2. Interface Inventory & CK Metrics

CK metrics (CBO, LCOM, RFC, WMC, DIT) are aggregated across implementing classes. NOC represents the number of concrete implementing classes.

Interface	Pkg	Health	Severity	Methods	CBO	LCOM	WMC	NOC
OrderService	`com.example.`	66.8	WARNING	8	1.0	0.00	7.0	2
BillingService	`com.example.`	96.2	HEALTHY	1	2.0	0.00	1.0	1

3. Interface Segregation Principle (ISP) Violations

Rule / Indicator	Class	Method	Evidence
Placeholder Stub Return	`FastTrackOrderService`	`cancelOrder`	Single return statement returns false suggests dummy implementation.
Exception-Throwing Stub	`FastTrackOrderService`	`refundPayment`	Throws new UnsupportedOperationException("FastTrack orders cannot be refunded") instead of providing required contract behavior.
Empty Implementation Method	`FastTrackOrderService`	`sendNotification`	Method body is completely empty, suggesting the class was forced to implement an unused method.
Exception-Throwing Stub	`FastTrackOrderService`	`generateInvoicePdf`	Throws new UnsupportedOperationException("PDF invoices not supported in fast-track") instead of providing required contract behavior.
Empty Implementation Method	`StandardOrderService`	`sendNotification`	Method body is completely empty, suggesting the class was forced to implement an unused method.
Placeholder Stub Return	`StandardOrderService`	`generateInvoicePdf`	Single return statement returns new byte[0] suggests dummy implementation.

4. Design Smells & Dependency Cycles

Category	Rule Name	Severity	Target	Details
Code Smell	**Fat Interface**	CRITICAL	`C:/athul/MCA/Sem 3/M`	Interface 'OrderService' declares 8 methods with low cohesive parameter sharing (75%).
Code Smell	**Refused Bequest**	HIGH	`FastTrackOrderServic e`	Implementing class 'FastTrackOrderService' ignores or rejects 4 of 8 interface methods.
Code Smell	**Refused Bequest**	HIGH	`StandardOrderService`	Implementing class 'StandardOrderService' ignores or rejects 2 of 8 interface methods.
Cycle	**Circular Dependency**	CRITICAL	`Cycle`	Cycle path: OrderService -> BillingService -> OrderService
Cycle	**Circular Dependency**	CRITICAL	`Cycle`	Cycle path: OrderService -> BillingService -> OrderService

5. Historical Evolution & Statistical Change Points

Commit	Date	Author	Score Drop	Description
`9cbd0a51`	2026-10-17	Demo Author	88.6 → 77.7	Health Score dropped by 10.9 points (from 88.6 to 77.7) in commit 9cbd0a5.

6. AI-Assisted Architecture & Refactoring Recommendations

Diagnosis (gemini-flash-lite-latest):

The 'OrderService' interface suffers from Interface Segregation Principle (ISP) violations and acts as a 'Fat Interface'. It forces implementing classes ('FastTrackOrderService' and 'StandardOrderService') to implement methods they do not fully support. This has led to code smells like stub returns, empty method implementations, and throwing 'UnsupportedOperationException' for methods such as 'cancelOrder', 'sendNotification', 'generateInvoicePdf', and 'refundPayment'.

Refactoring Plan & Responsibility Separation:

1. Analyze the core capabilities required by different order types. 2. Extract specific, cohesive subsets of the methods ('cancelOrder', 'sendNotification', 'generateInvoicePdf', 'refundPayment') into smaller interfaces. 3. Update 'FastTrackOrderService' and 'StandardOrderService' to implement only the granular interfaces relevant to their specific business logic. 4. Refactor client code to depend on the newly segregated interfaces rather than the monolithic 'OrderService'.

Risks & Trade-offs: Refactoring a core interface like 'OrderService' requires updating all upstream client dependencies and injection configurations. Breaking changes will occur if clients expect the monolithic interface, so careful deprecation and incremental migration strategies should be employed.

Diagnosis (deterministic_expert_engine):

Interface 'BillingService' exhibits design erosion symptoms. Implementing classes are forced to provide 6 empty stubs or UnsupportedOperationException stubs, directly violating the Interface Segregation Principle.

Refactoring Plan & Responsibility Separation:

Apply the 'Extract Interface' refactoring pattern: - 'BillingService': extract [invoiceCustomer] - Have implementing classes implement only the specific interfaces they actually fulfill.

Risks & Trade-offs: Refactoring interface contracts introduces temporary breaking changes for external callers. Deprecate bloated methods first, provide default adaptor implementations, and migrate client call sites incrementally.

7. Methodology & Known Limitations

Methodology: InterfaceGuard analyzes Java source code across Git history using JavaParser AST parsing and PyDriller. Metric extraction includes Chidamber & Kemerer (CK) metrics aggregated over implementing classes, interface cohesion metrics, ISP violation heuristics (empty/no-op/exception/placeholder stubs), and NetworkX cycle analysis.

Limitations: Dynamic runtime reflection and bytecode manipulation are out of scope. Statistical change-point detection highlights sudden score deteriorations but does not prove intent. All recommendations should be reviewed by engineering teams.